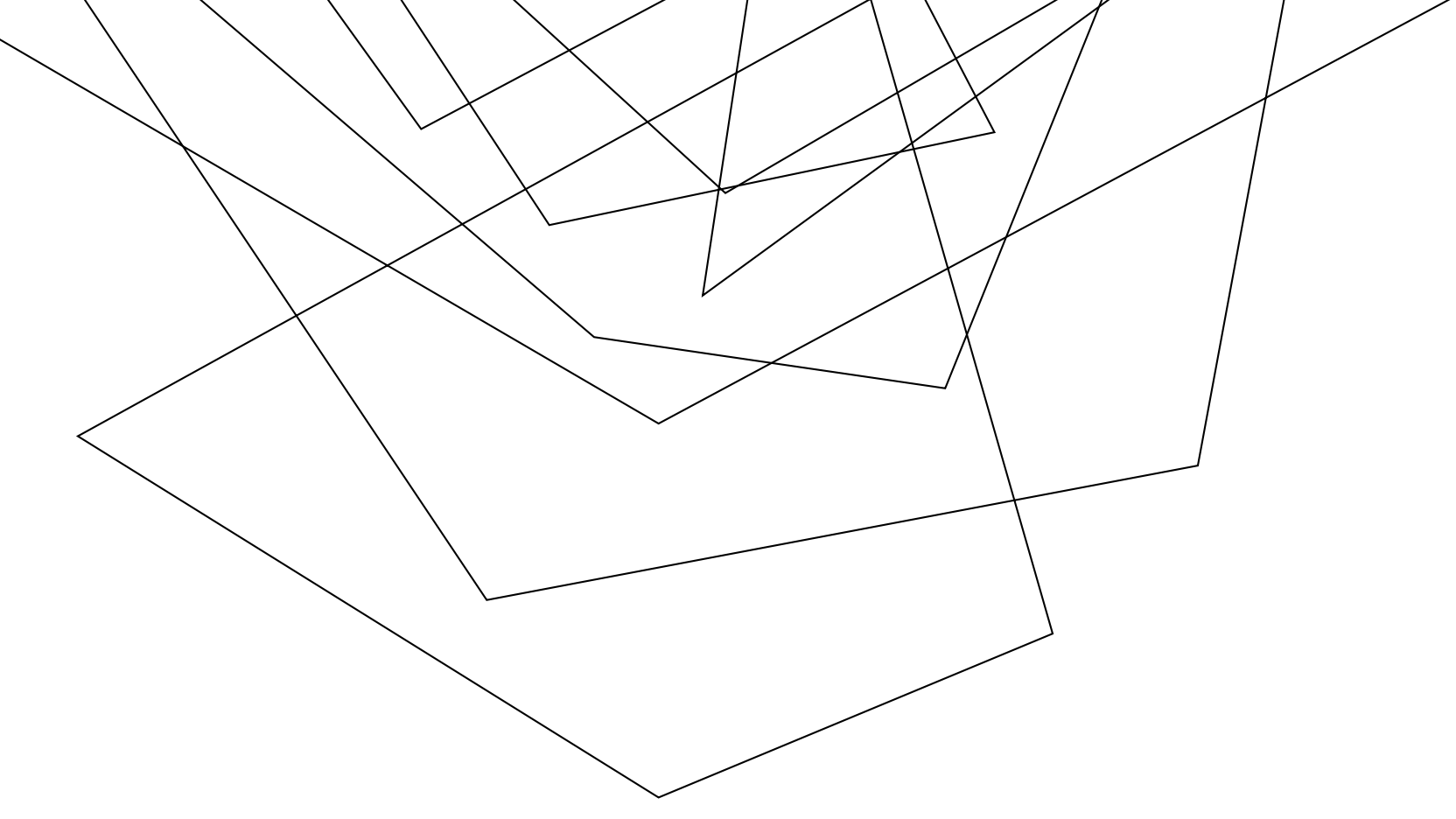




**Hof
University**
University of
Applied Sciences



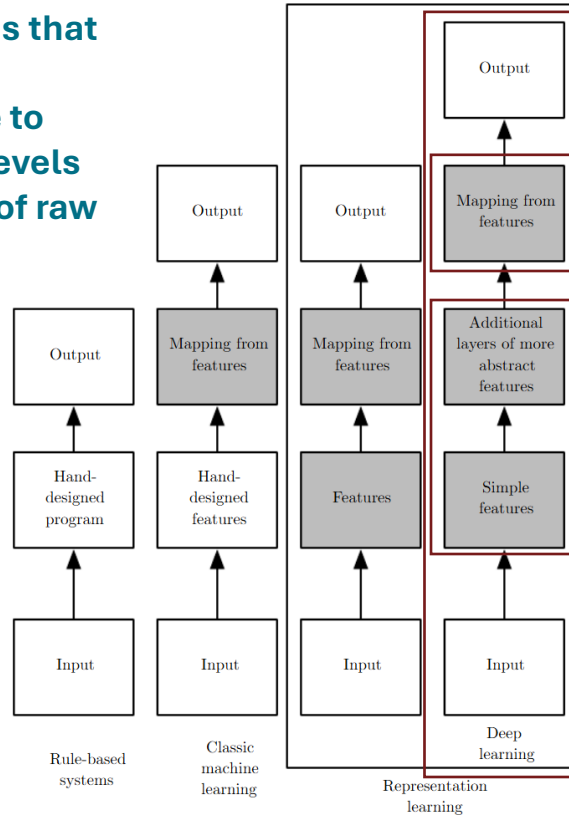
APPLIED DEEP LEARNING

05 – CONVOLUTIONAL NEURAL NETWORKS (CNNs)

PROF. DR. LARS ACKERMANN-IGL

RECAP: ADVANTAGES OF DEEP LEARNING – USING ARTIFICIAL NEURAL NETWORKS

One advantage is that **Deep-Learning models are able to learn different levels of abstractions of raw features ...**



OPEN QUESTIONS

How to **USE** deep learning? We so far only explained rather abstract principles?

How are features mapped?

How do deep-learning models learn feature abstractions?



All answered for working with image data in this slide set!

Figure 1.5: Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data.



LIMITATIONS OF MLPs (a limited assessment)

- Processing Image data with MLPs
- Curse of the dimensionality

MOTIVATION: PROBLEMS INVOLVING IMAGE DATA

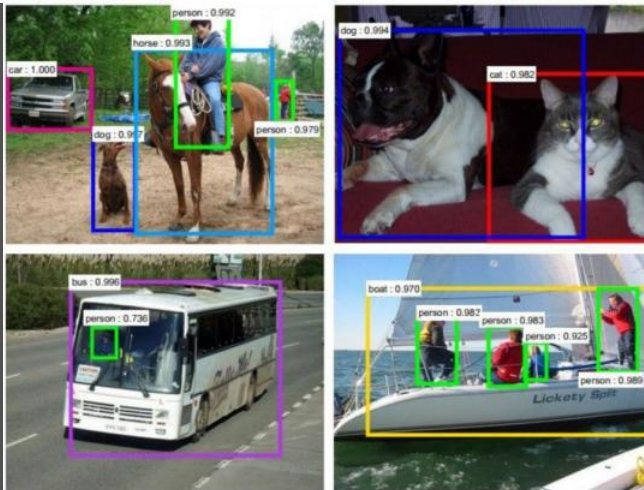
Classification



Retrieval



Detection



Segmentation



Many real-world applications require the usage of image information, so the interesting question is how can we utilize image information for our problems?

More specifically:
How can we utilize image information if we want to do e.g., image classification using our currently known MLP approach?

WHAT MAKES WORKING WITH IMAGE DATA CHALLENGING

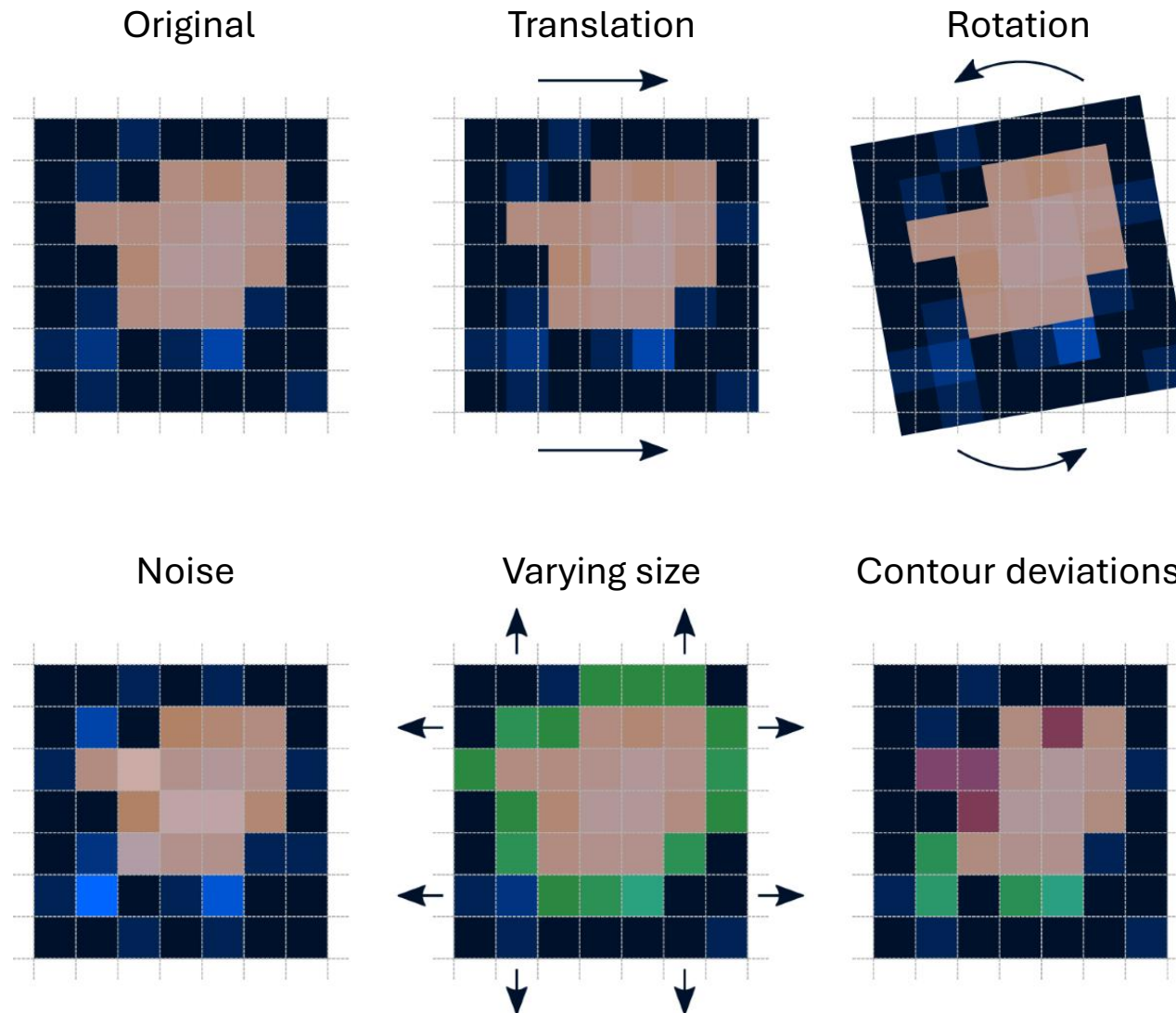


IMAGE CLASSIFICATION WITH MLPs – 1

APPLICATION: IMAGE CLASSIFICATION

Assume we have some pre-labeled data with discrete labels e.g. $Y = \{cat, dog, ship, truck, \dots\}$:

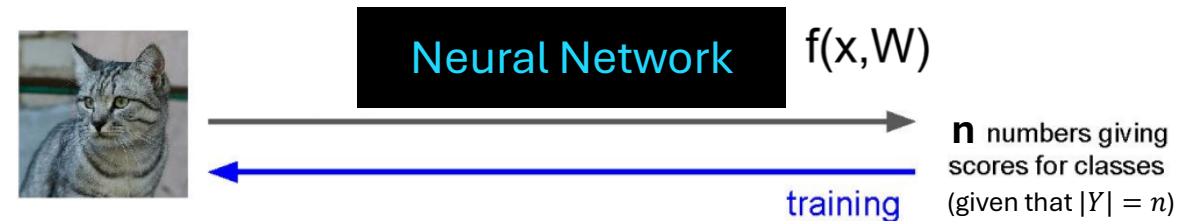
$$f(\text{img}_{cat}) = "cat"$$

$$f(\text{img}_{grumpy_cat}) = "cat"$$

$$f(\text{img}_{ship}) = "ship"$$

How can we define this $f(x)$?

SOLVING THE PROBLEM WITH MLPs:



Recap: By utilizing neural network-based approaches, we **want to get rid** of the (manual) **feature selection and extraction process**, this means the neural network **should learn the relevant features** for a specific class **by itself utilizing available labeled data**.



Idea: Use each pixel of the input image as **potential feature** and utilize **hidden layers** to specify more complex / abstract features (think back to our NN which is able to model an XOR)

IMAGE CLASSIFICATION WITH MLPs – 2

APPLICATION: IMAGE CLASSIFICATION

Assume we have some pre-labeled data with discrete labels e.g., $Y = \{cat, dog, ship, truck, \dots\}$:

$$f(\text{Image of a cat}) = \text{"cat"}$$

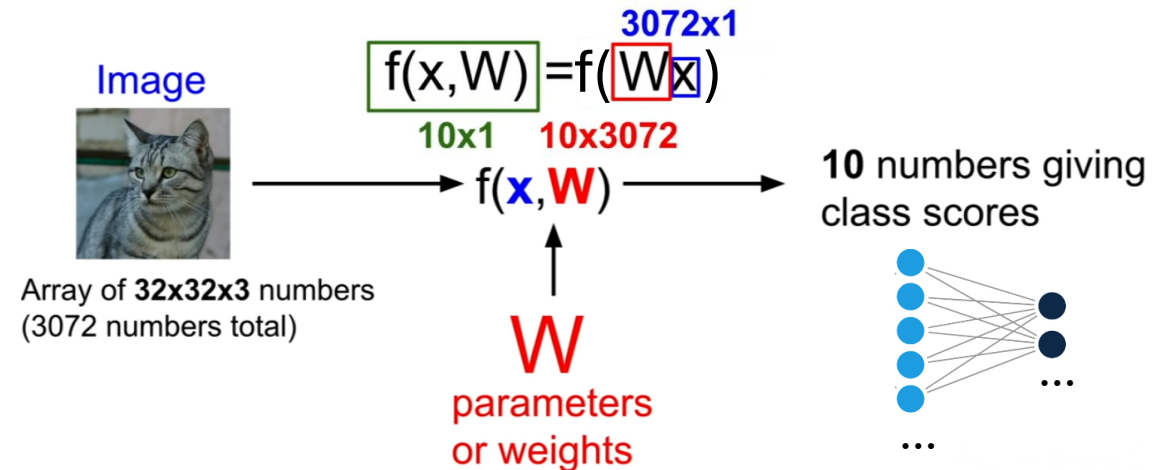
$$f(\text{Image of Grumpy Cat}) = \text{"cat"}$$

$$f(\text{Image of a ship}) = \text{"ship"}$$

How can we define this $f(x)$?

SOLVING THE PROBLEM MLPs:

By using the proposed main idea, what does this mean in terms of weight parameters (when using e.g., input images with resolution 32x32 and classify in total 10 classes)?



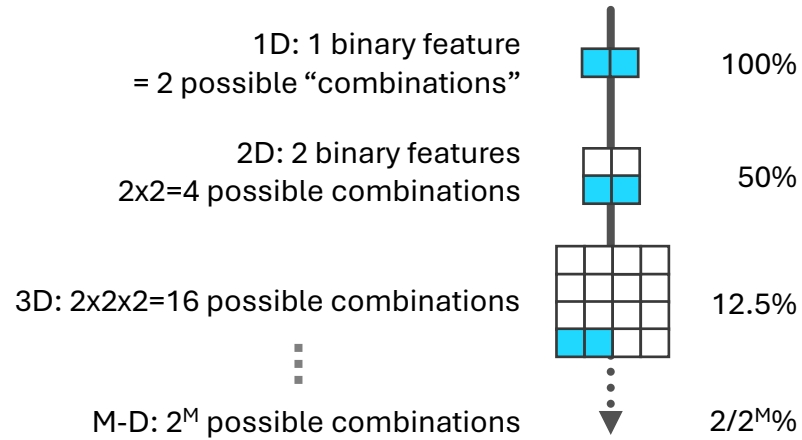
Large number of parameters need to be specified!

Think about what will happen if we want to use larger input images or we want to define a more complex network architecture consisting of multiple hidden layers.

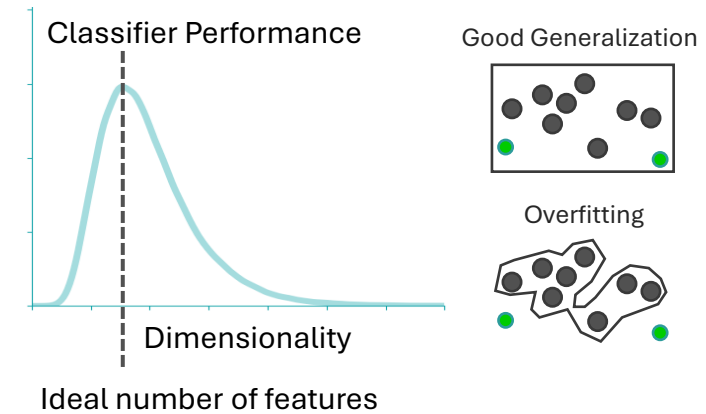
ISSUE IS KNOWN AS: CURSE OF THE DIMENSIONALITY

The amount of data required for good performance of machine learning algorithms increases exponentially with increasing dimensionality.

SPARSE DATA



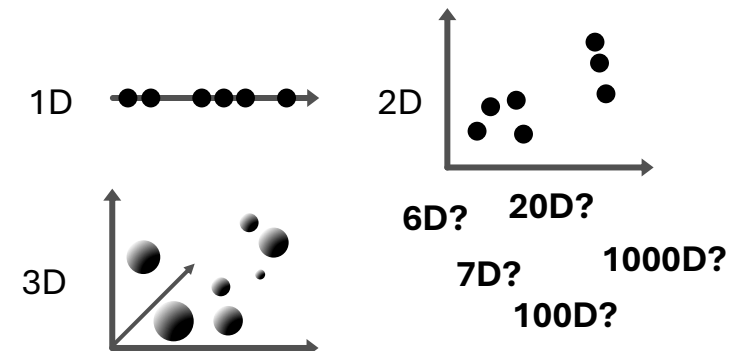
PEAKING/HUGHES' PHENOMENON



INCREASING COMPUTATION TIMES



VISUALIZATION ISSUES



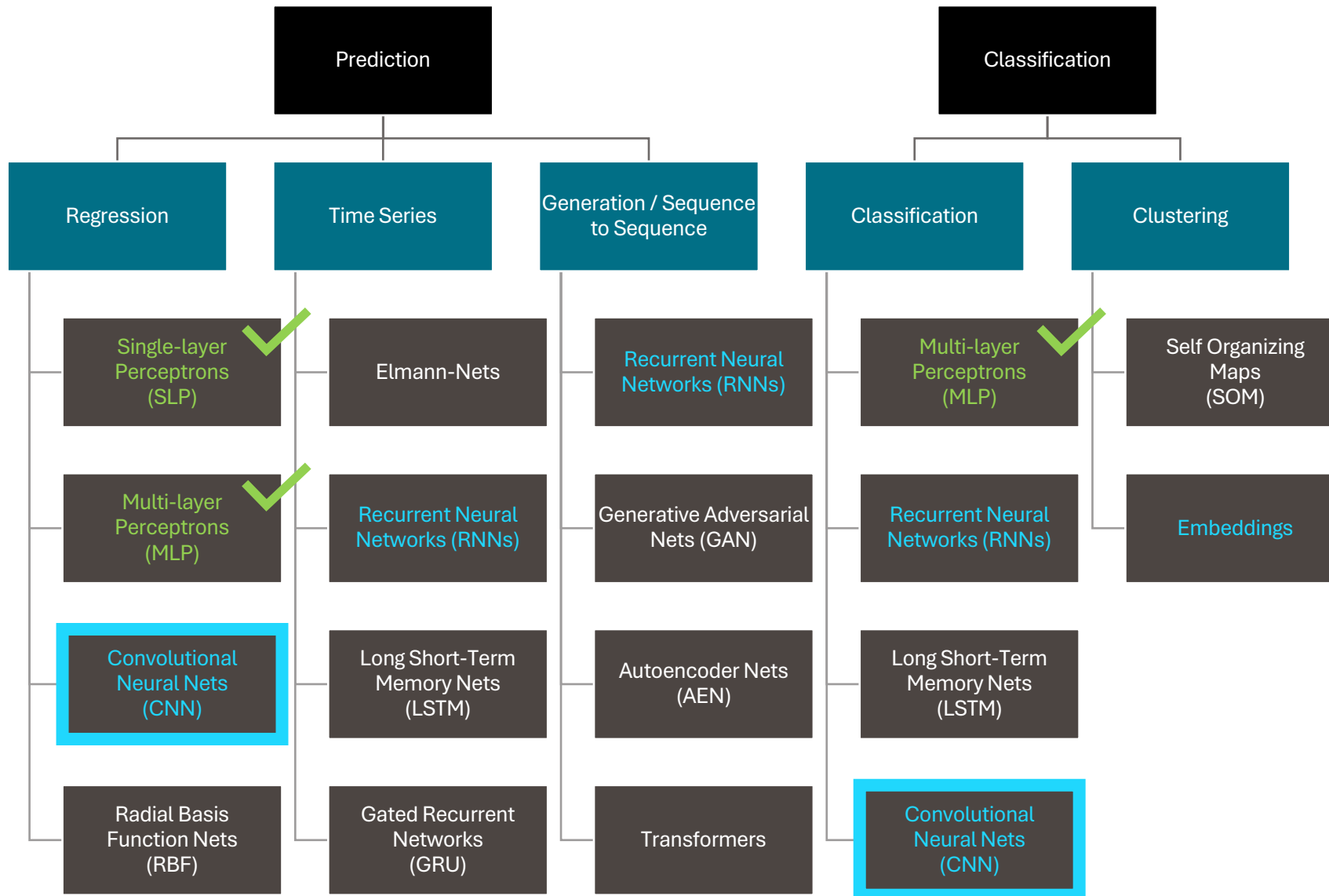
SO, WHAT TO DO?

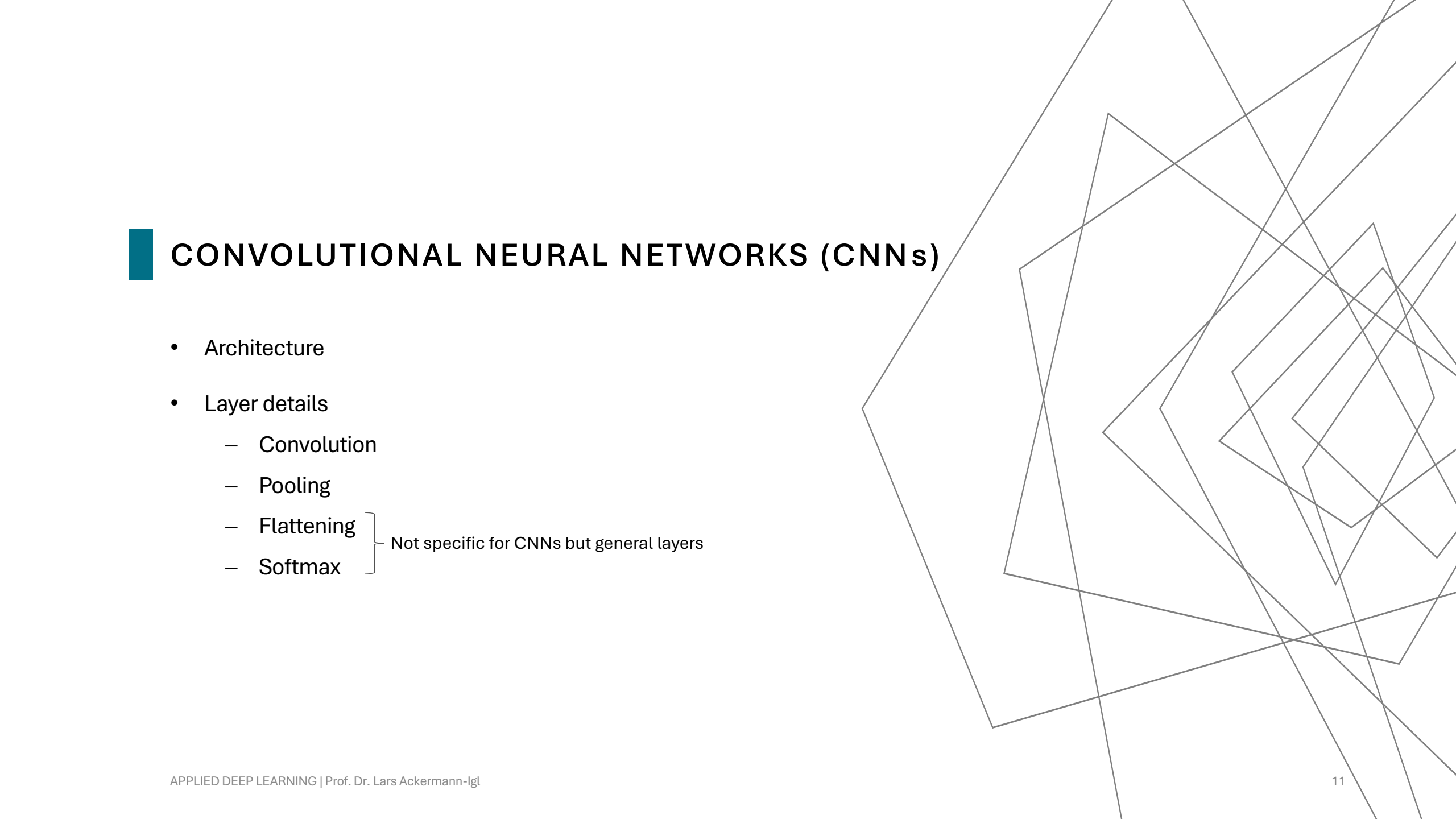
By utilizing NNs, we might be able to get rid of the (manual) **feature selection and extraction process**.

When our NN should be able use **each pixel of the input image as a potential feature** and it should be able to **identify more complex features / describe more complex decision boundaries**, then we typically need a massive amount of training data (since we have a massive number of parameters to be trained) → cf. Curse of the Dimensionality.

**Can we make some adjustments in our neural network architecture
in order to optimize this observation?**

THIS IS WHERE CNNs COME INTO PLAY

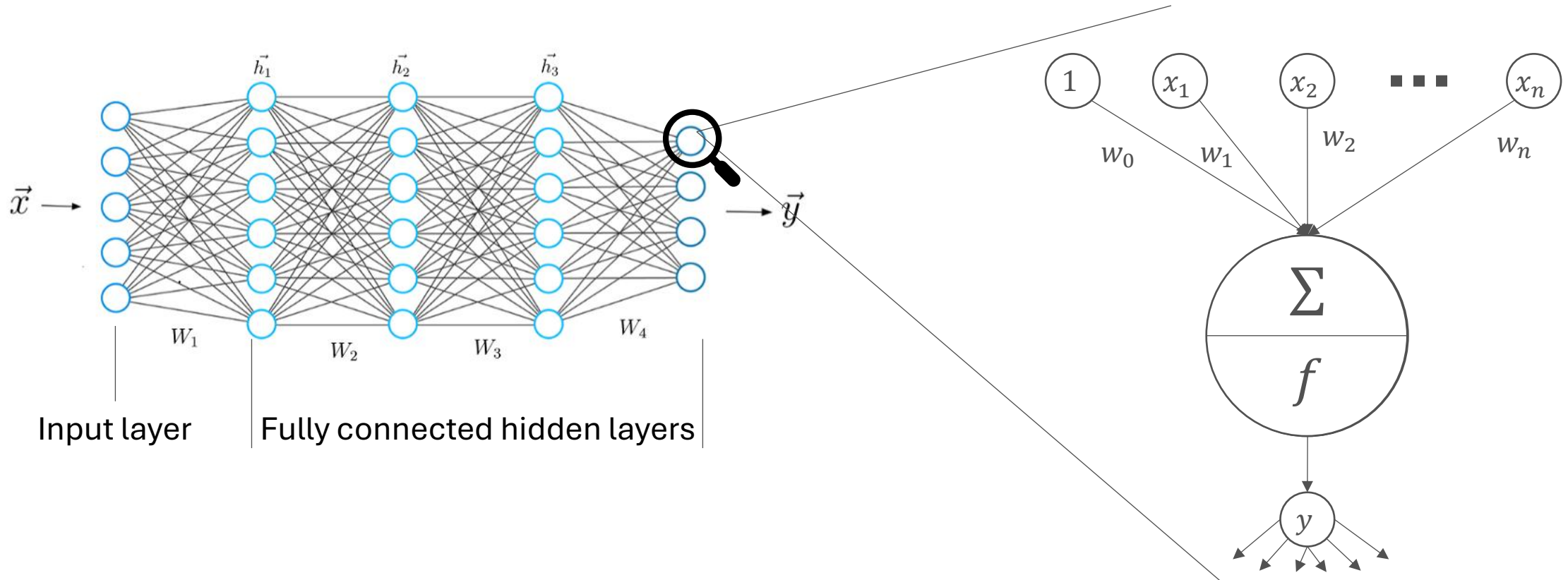




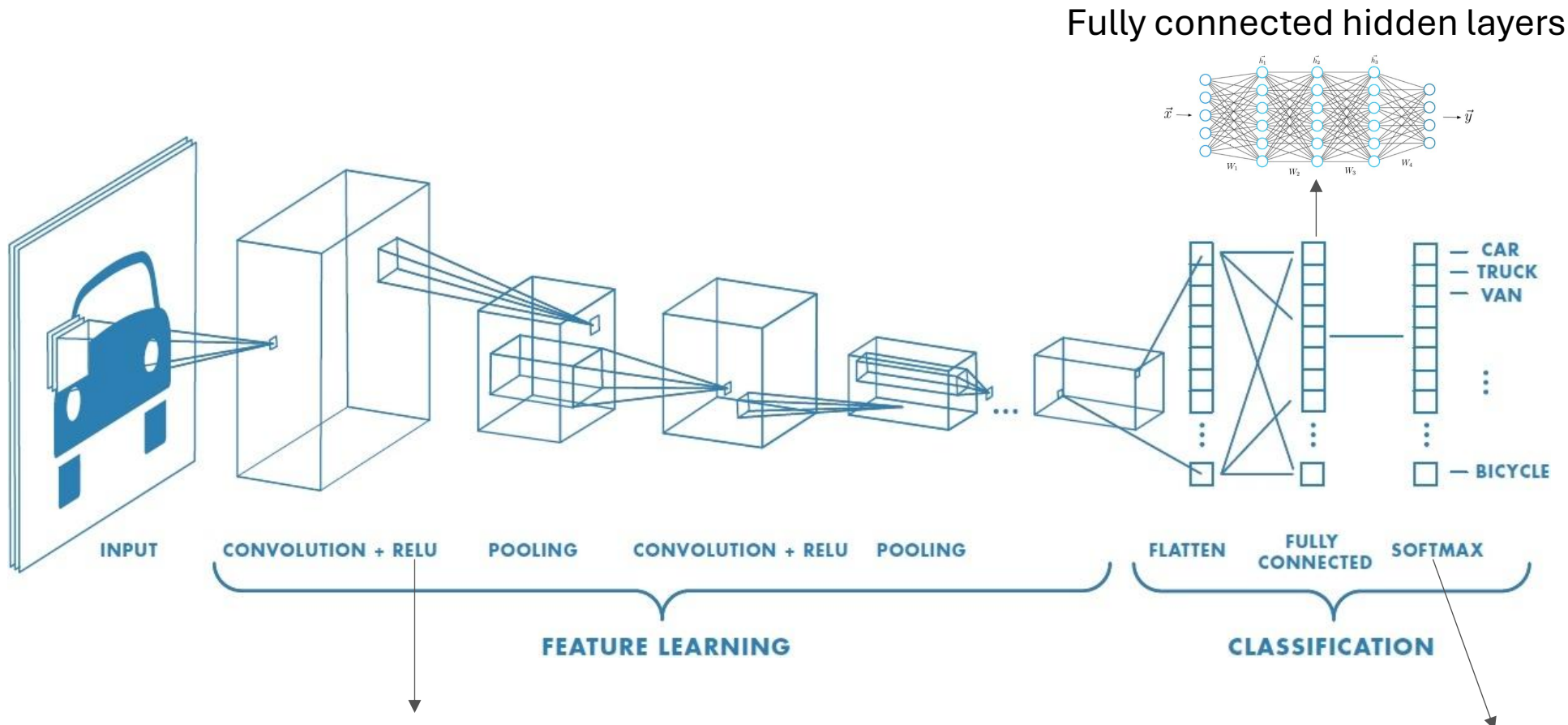
CONVOLUTIONAL NEURAL NETWORKS (CNNs)

- Architecture
 - Layer details
 - Convolution
 - Pooling
 - Flattening
 - Softmax
- } Not specific for CNNs but general layers

RECAP: ARCHITECTURE OF MLPs



CNNs – ARCHITECTURE



Actually, this can be an arbitrary activation function but most of the time it is the identity function or the ReLU!

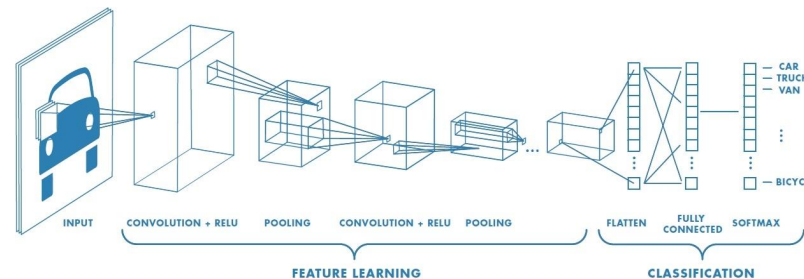
When used for classification; for regression the output layer is just a fully connected layer

CNNs – ARCHITECTURE

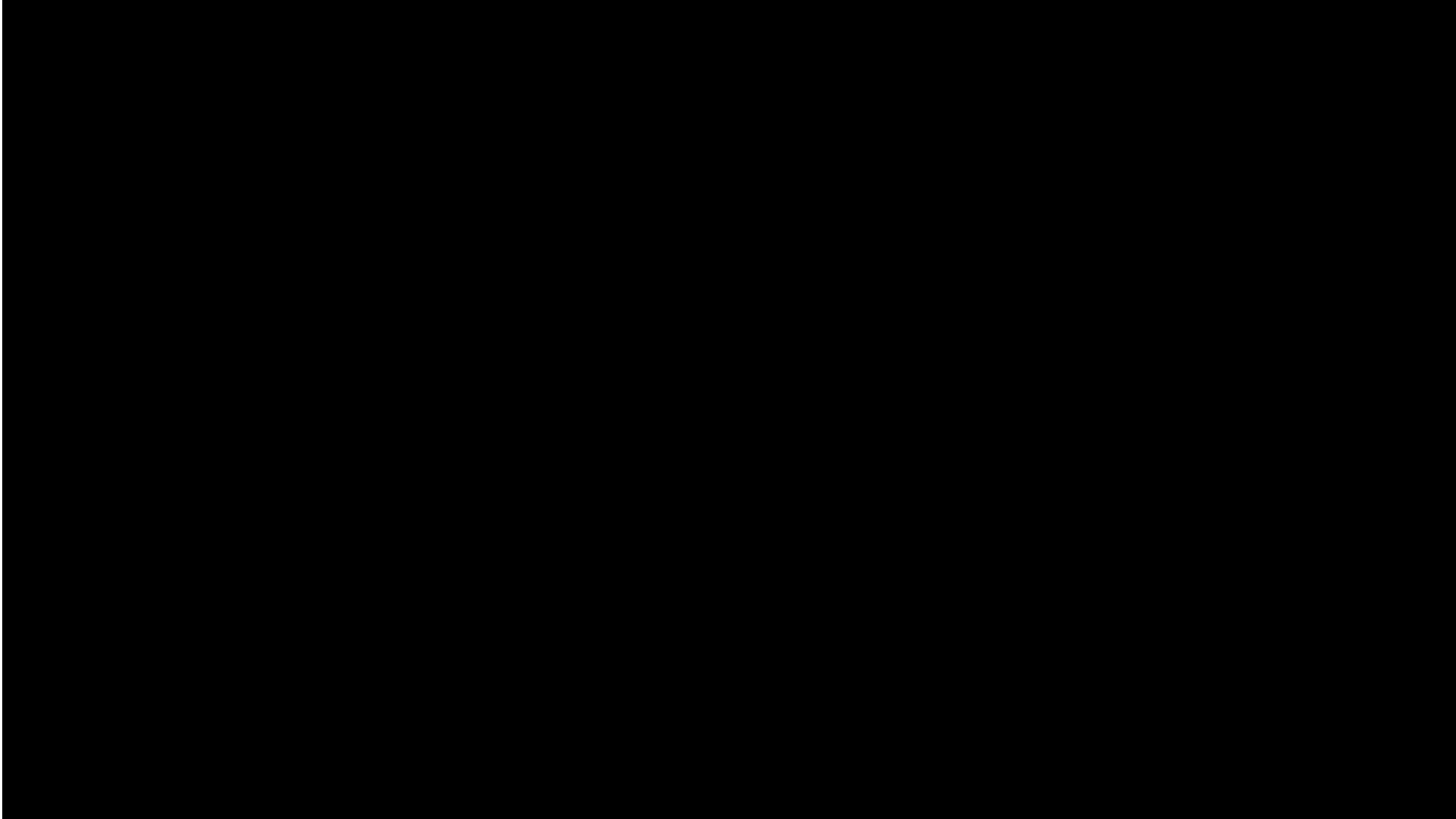
CONTENTS OF THE ARCHITECTURE

- **Input layer**
 - Accepts the raw data input
- **Feature detection layers**
 - **Convolution** puts the input images through a set of **convolutional filters**, each of which activates certain features from the images. → Learns feature representations of the input
 - **Pooling** simplifies the output by performing nonlinear downsampling, reducing the number of parameters that the network needs to learn about (max pooling, min pooling, avg pooling, ...).
 - Combining convolutions with **Activation** (e.g. ReLU) allows for faster/more effective training by mapping negative values to zero and preserving positive values.
- **Classification layers (flattening and fully connected layers are also present in regression scenarios)**
 - **Flattening** the resulting feature maps into a one-dimensional vector so they can be passed into a completely linked layer for categorization or regression.
 - The next-to-last layer is a **fully connected layer (FC)** that outputs a vector of K dimensions where K is the number of classes that the network will be able to predict. This vector contains the probabilities for each class of any image being classified.
 - The **output** layer of the CNN architecture uses a classification function (e.g., SOFTMAX) function to provide the classification output.

Filters? See next slide!

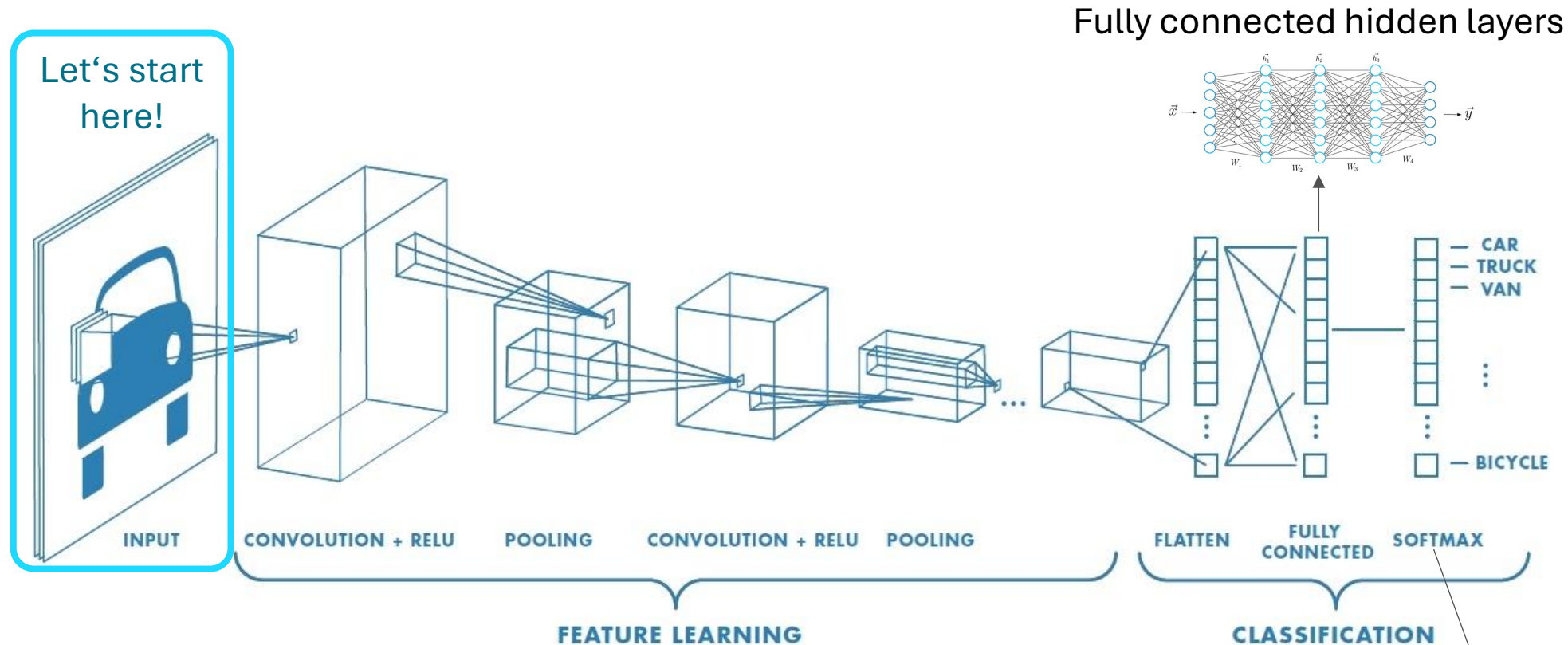


MOVIE TIME – WHY FILTERS ARE USEFUL



<https://www.youtube.com/watch?v=QzY57FaENXg>

STEP BY STEP: THE INPUT



When used for classification;
for regression the output layer is
just a fully connected layer

STEP BY STEP: THE INPUT

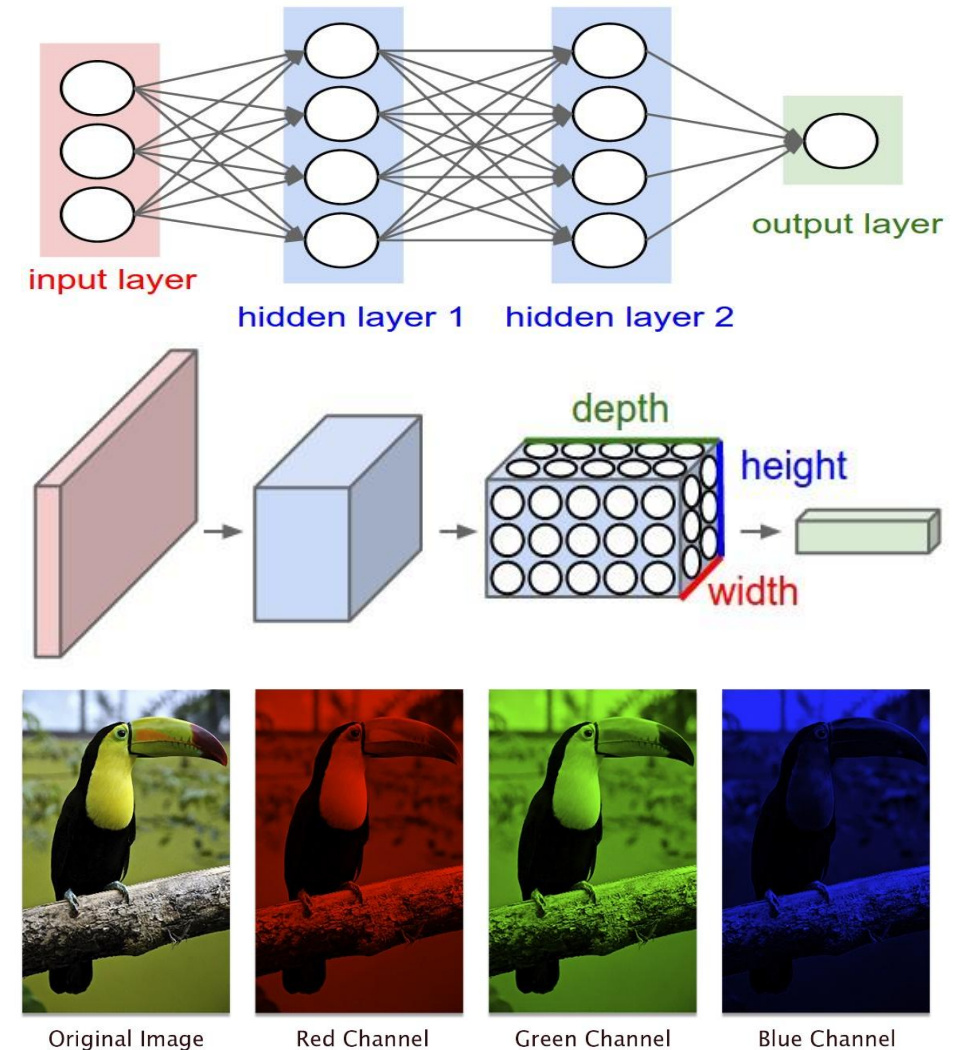
PROBLEM

Our currently used NNs take only a **feature vector** as input. This means when we want to use an image as input, we need to perform some form of “**flattening**” of the image data (e.g. **column- or row-wise**). By doing so, we might lose information about **local connectivity**, which might be relevant for our task.



IDEA

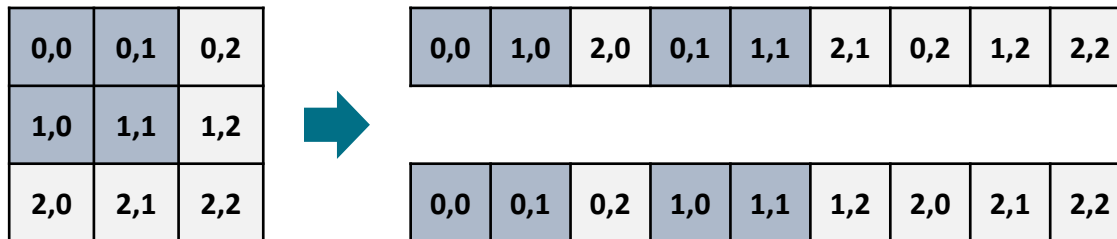
Use the images **directly** as input, without doing the flattening task. Since we typically deal with multicolor images (e.g. RGB images), **process the image as 2D slices of a 3D volume** (image **width**, image **height** and the amount of channels as **depth**).



STEP BY STEP: THE INPUT

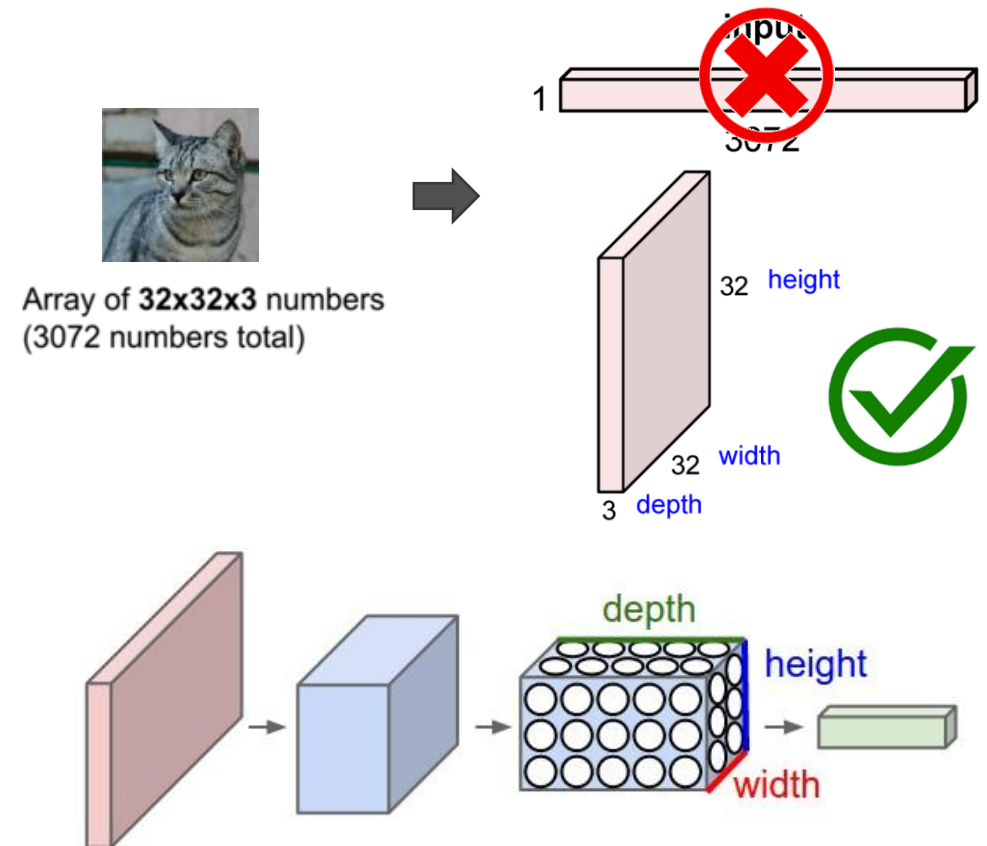
PROBLEM

Our currently used NNs take only a **feature vector** as input. This means when we want to use an image as input, we need to perform some form of “**flattening**” of the image data (e.g. **column- or row-wise**). By doing so, we might lose information about **local connectivity**, which might be relevant for our task.



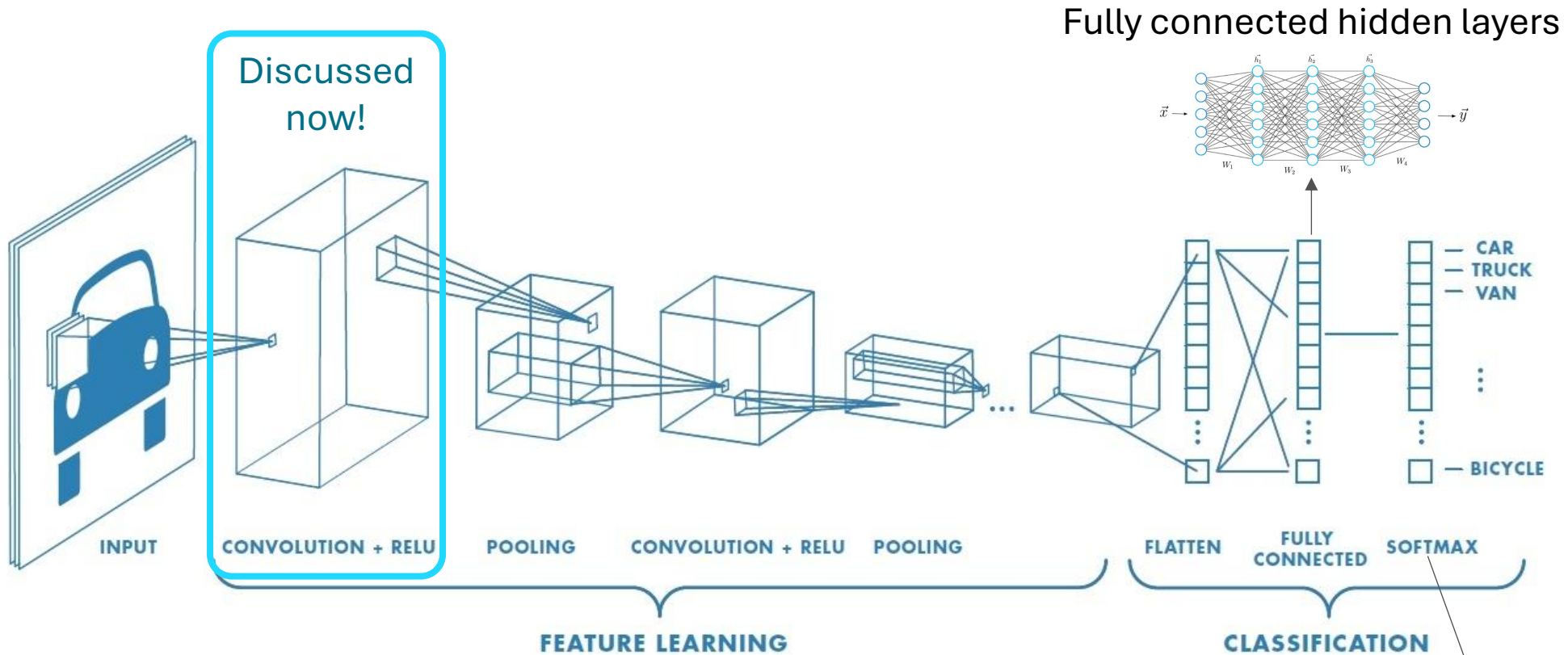
IDEA

Use the images **directly** as input, without doing the flattening task. Since we typically deal with multicolor images (e.g. RGB images), **process the image as 2D slices of a 3D volume** (image **width**, image **height** and the amount of channels as **depth**).



How is the image data further processed?

STEP BY STEP: CONVOLUTIONS

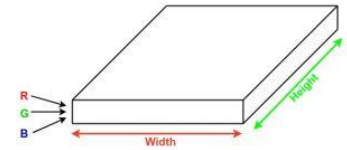


When used for classification;
for regression the output layer is
just a fully connected layer

STEP BY STEP: CONVOLUTION LAYERS – OVERVIEW

PROBLEM

Since we currently have only **raw pixel** values, we want to define something, which is able to **calculate potential relevant features** in our input image. Baseline features might be **edges, corners**, or **color gradients**, and so on.



IDEA

Utilize a set of **filter (convolutional kernels)** in order to calculate **potentially relevant features** in the input images.

- **Convolution kernels (filters)**

- Is a small **pixel mask / matrix** (typical sizes are e.g. 1x1, 3x3, 5x5, 7x7, 11x11)
- **Applied** to the input image in order to **calculate a new image**
- Task: extract meaningful features
- These masks are trainable parameters, i.e., values are assigned randomly at the beginning



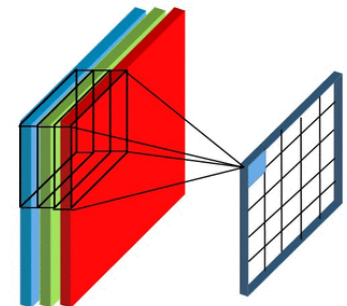
These parameters are out trainable weights!

- **Convolution operation** (detailed on next slides)

- Take the kernel and slide it over the complete image horizontally as well as vertically
- Output: **feature map (convolved feature)**
- From a mathematical perspective, the output of this operation is the **convolution** between a function i (our **input image**) and a function k (our **filter / convolution kernel**), typically denoted by $i * k$.

2 x 2 kernel

0	1
-1	2



CHECK THIS OUT!

Image Kernels

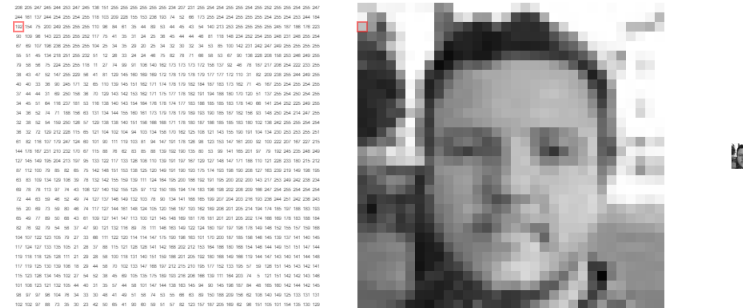
Explained Visually

% Post

By [Victor Powell](#)

An image kernel is a small matrix used to apply effects like the ones you might find in Photoshop or Gimp, such as blurring, sharpening, outlining or embossing. They're also used in machine learning for 'feature extraction', a technique for determining the most important portions of an image. In this context the process is referred to more generally as "convolution" (see: [convolutional neural networks](#)).

To see how they work, let's start by inspecting a black and white image. The matrix on the left contains numbers, between 0 and 255, which each correspond to the brightness of one pixel in a picture of a face. The large, granulated picture has been blown up to make it easier to see; the last image is the "real" size.



Let's walk through applying the following 3x3 **sharpen** kernel to the image of a face from above.

sharpen

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

<https://setosa.io/ev/image-kernels/>

STEP BY STEP: CONVOLUTION LAYERS – COMPUTATION

COMPUTATION PRINCIPLE

Take the dot product between kernel and the **receptive fields** of the input image by multiplying the corresponding values of them and sum up all values to generate one scalar value in the output feature map

→ checks whether a feature is present in the image

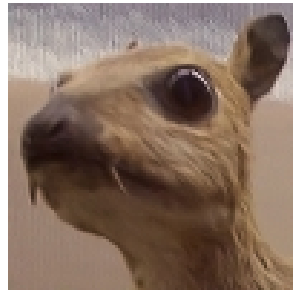
$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i + m, j + n) \cdot K(m, n)$$

- $I(i, j)$: The pixel value at position (i, j) in the input image.
- $K(m, n)$: The weight of the kernel at position (m, n) .
- $S(i, j)$: The output feature map at position (i, j) .

The **receptive field** is the region of the input image that a particular neuron in a convolutional layer is influenced by or "sees." It is defined by the position of the kernel window.

EXAMPLE (IGNORING THE CHANNELS FOR NOW)

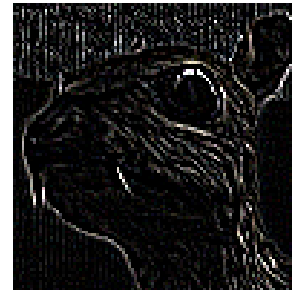
Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
...						

*

a	b	c
d	e	f
g	h	i

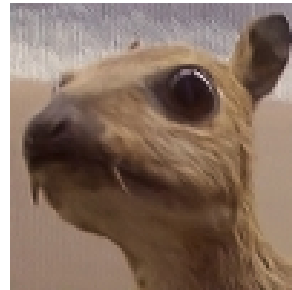
■ = $0a + 1b + 2c + 7d + \dots + 16i$ ←

■ = ?

■ = ?

EXAMPLE (IGNORING THE CHANNELS FOR NOW)

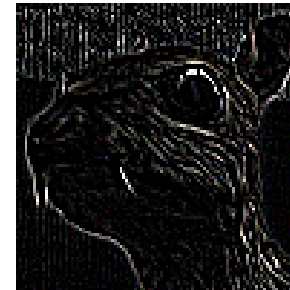
Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

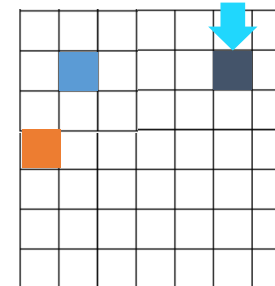
Feature map



0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
...						

*

a	b	c
d	e	f
g	h	i



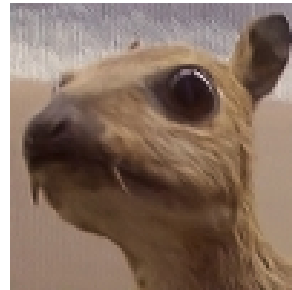
■ = $0a + 1b + 2c + 7d + \dots + 16i$

■ = $4a + 5b + 6c + 11d + \dots + 20i$ ←

■ = ?

EXAMPLE (IGNORING THE CHANNELS FOR NOW)

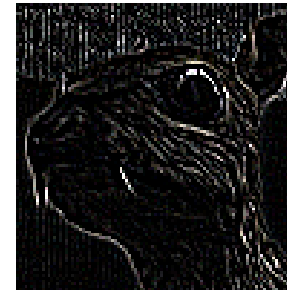
Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
...						

*

a	b	c
d	e	f
g	h	i

■ = $0a + 1b + 2c + 7d + \dots + 16i$

■ = $4a + 5b + 6c + 11d + \dots + 20i$

■ = ?



?

How to handle border pixels???

?

What does it mean “moving the window”?

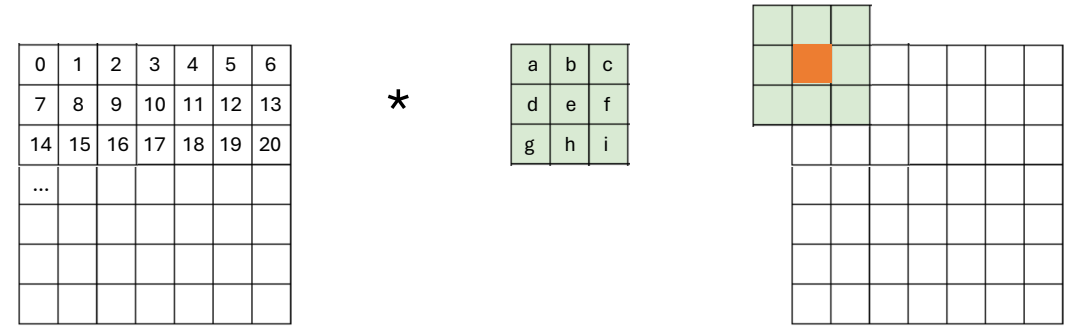
STEP BY STEP: CONVOLUTION LAYERS – PADDING

PROBLEM: BORDER PIXELS

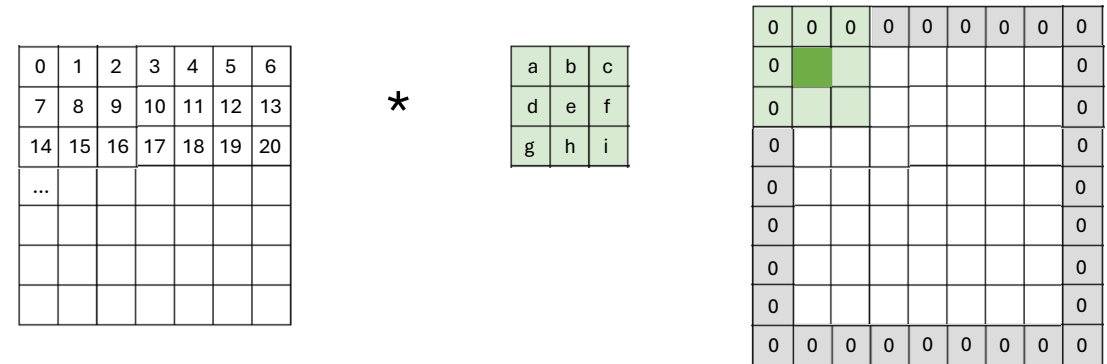
As we have seen before, when we want to calculate the convolution between an image and a filter, **border pixels might be a problem**.

IDEA

Define a parameter **padding**, which describes how many border pixels should be inserted and filled with 0.



Without padding



With padding

STEP BY STEP: CONVOLUTION LAYERS – STRIDE

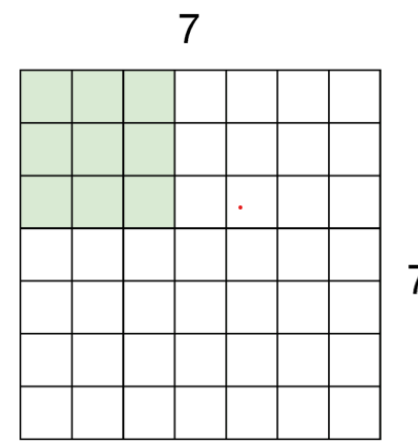
PROBLEM: SLIDING-WINDOW APPROACH

Since we want to **consecutively apply our filter** on the input image, we might also want to define the “**step-size**” for each iteration.

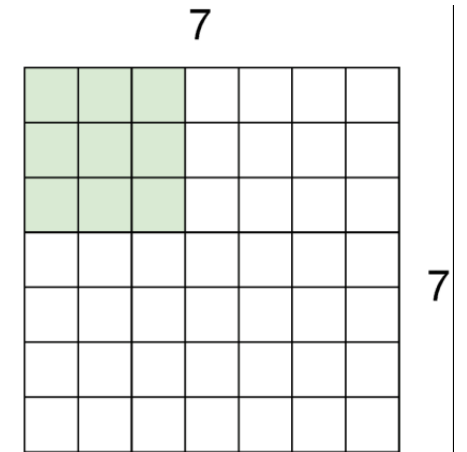
IDEA

Define a parameter **stride**, which describes for **how many pixels the filter is shifted** for the next iteration.

Imagine we have a 7x7 input image and want to apply a 3x3 filter consecutively.



stride = 1
output = 5x5



stride = 2
output = 3x3

?

What will happen for stride = 3?

STEP BY STEP: CONVOLUTION LAYERS – PARAMETER DEPENDENCIES/CORRELATIONS

PROBLEM: PARAMETER DEPENDENCIES/CORRELATIONS

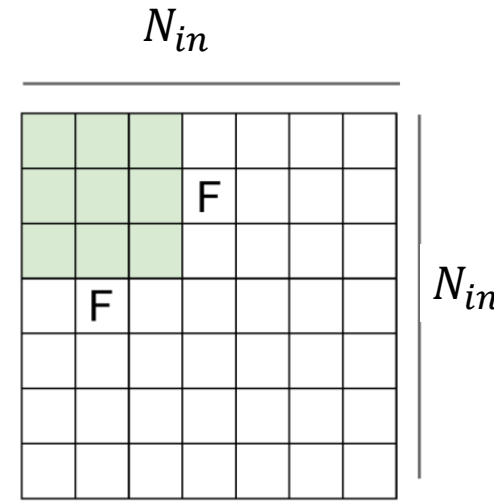
In general, we can observe a **correlation** between our introduced parameters!

CONSEQUENCE

Not all parameter combinations might be suitable (since output size must be an integer; however, the fallback is simply to only consider “full steps”) → adjust padding

?

Question: How can we utilize this in our neural network?



Output size

N_{in} ... Input size
 F ... Kernel size
 P ... Padding
 N_{out} ... Output size
 s ... Stride

$$N_{out} = \frac{N_{in} + 2P - F}{s} + 1$$

Example:

$$N_{in} = 7, F = 3, P = 0$$

$$\text{stride } 1 \Rightarrow (7 - 3)/1 + 1 = 5 \quad \checkmark$$

$$\text{stride } 2 \Rightarrow (7 - 3)/2 + 1 = 3 \quad \checkmark$$

$$\text{stride } 3 \Rightarrow (7 - 3)/3 + 1 = 2.33 \quad \times$$

!

For non-squared images, the output size is calculated for each dimension separately!

STEP BY STEP: CONVOLUTION LAYERS – HOW TO USE CONVOLUTIONS IN NEURAL NETWORKS

“PROBLEM”: CONVOLUTIONS ARE NO NEURONS

We know fully connected layers only. How can convolutions be integrated?

IDEA

Instead of using a **fully connected network / layer**, utilize the concept of **convolution** in order to calculate relevant features. Define the output of a **single iteration of the overall convolution as a neuron**.

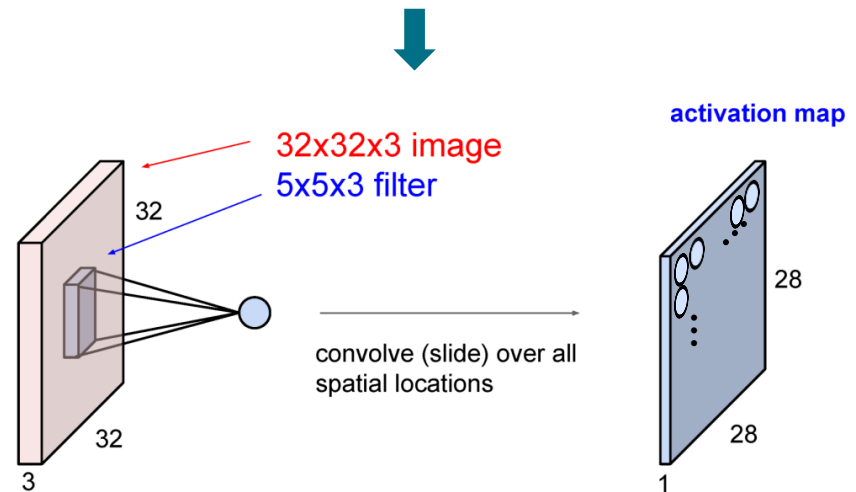
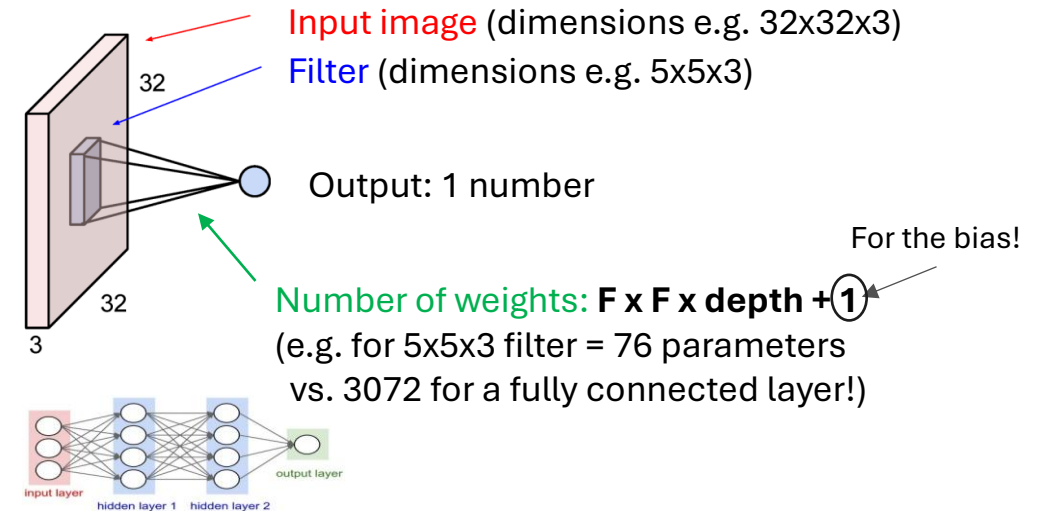
!

This type of neurons form a convolutional layer in our CNN architecture.

?

How many weights need to be trained if we e.g. want to use a 32x32x3 image as input, and one 5x5x3 filter (resulting in 28x28x1 neurons when not using padding)?

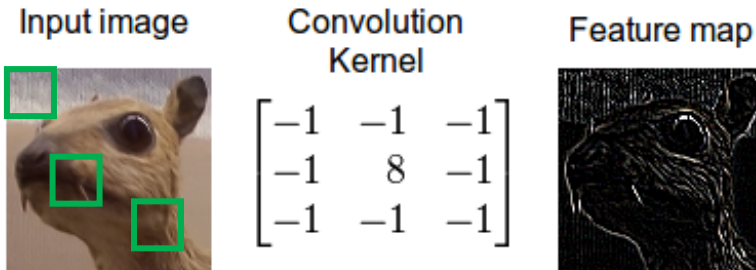
Hint: It is not $(5 \times 5 \times 3 + 1)(28 \times 28)$



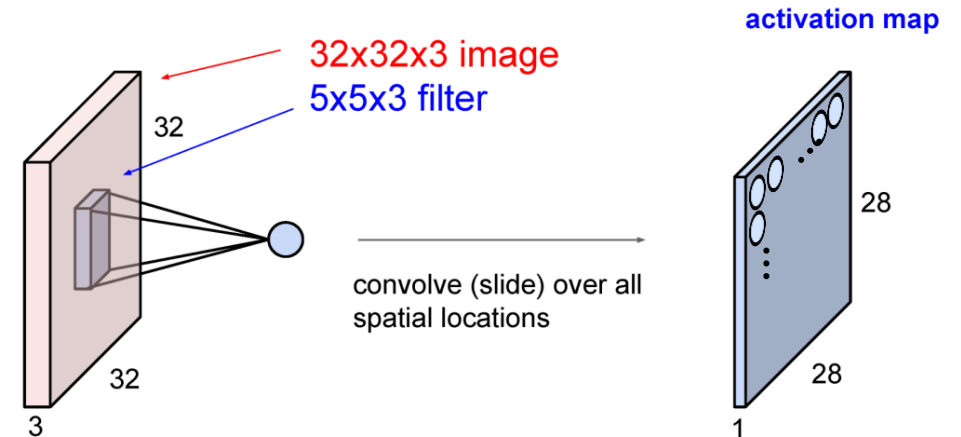
WHY IS THE NUMBER OF PARAMETERS SO LOW?

REASON

Since we can define a **filter for extracting a specific feature** (e.g. an edge in an image), we **assume** that the same filter is able to extract the specific feature **for different inputs / applied image positions**.



Filter weights are always the same, only the **position** of the input image is varied. Therefore, we **need to train the weights for a specific filter only once**, all other neurons (applying the same filter) **share the parameter weights**. This is called the concept of **parameter sharing**.



Since we can utilize the concept of parameter sharing, we only need to train $F \times F \times \text{depth} + 1$ parameters (76 in our example).

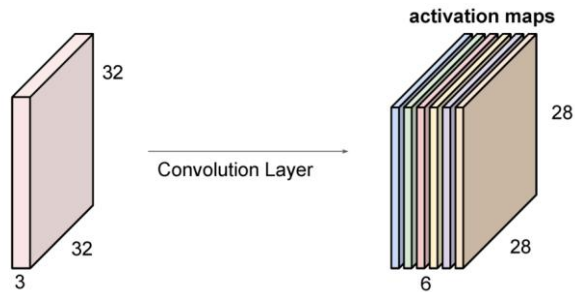
This means we can dramatically reduce the amount of trainable parameters.

In practice, we typically increase the number of filters per CONV-layer (which are able to detect specific features)

STEP BY STEP: CONVOLUTION LAYERS – MULTIPLE FILTERS

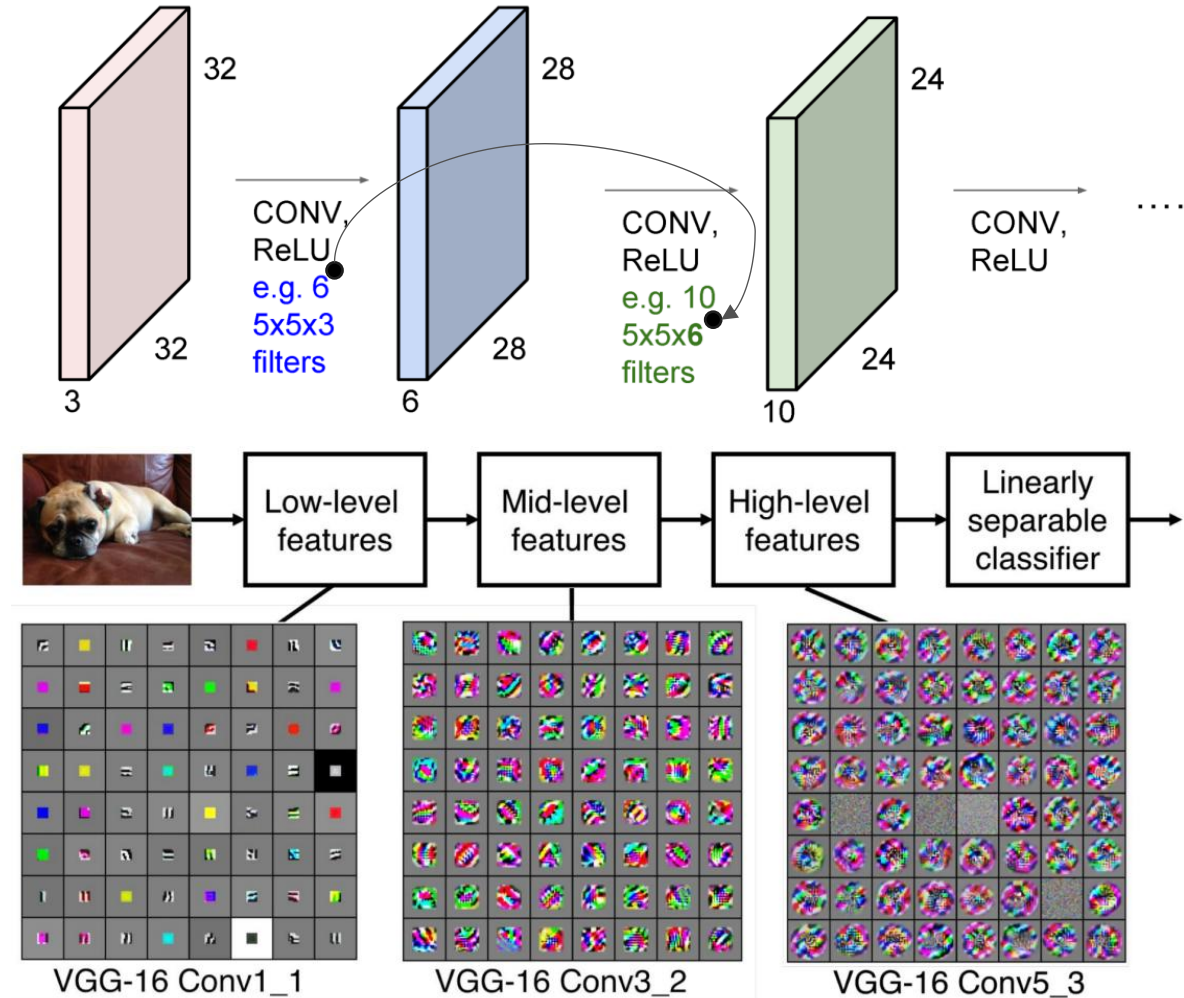
MULTIPLE FILTERS

As mentioned, in practice we typically use **multiple filters** **per convolutional layer**, in the example we use e.g. 6 different 5x5x3 filters in one convolution layer.

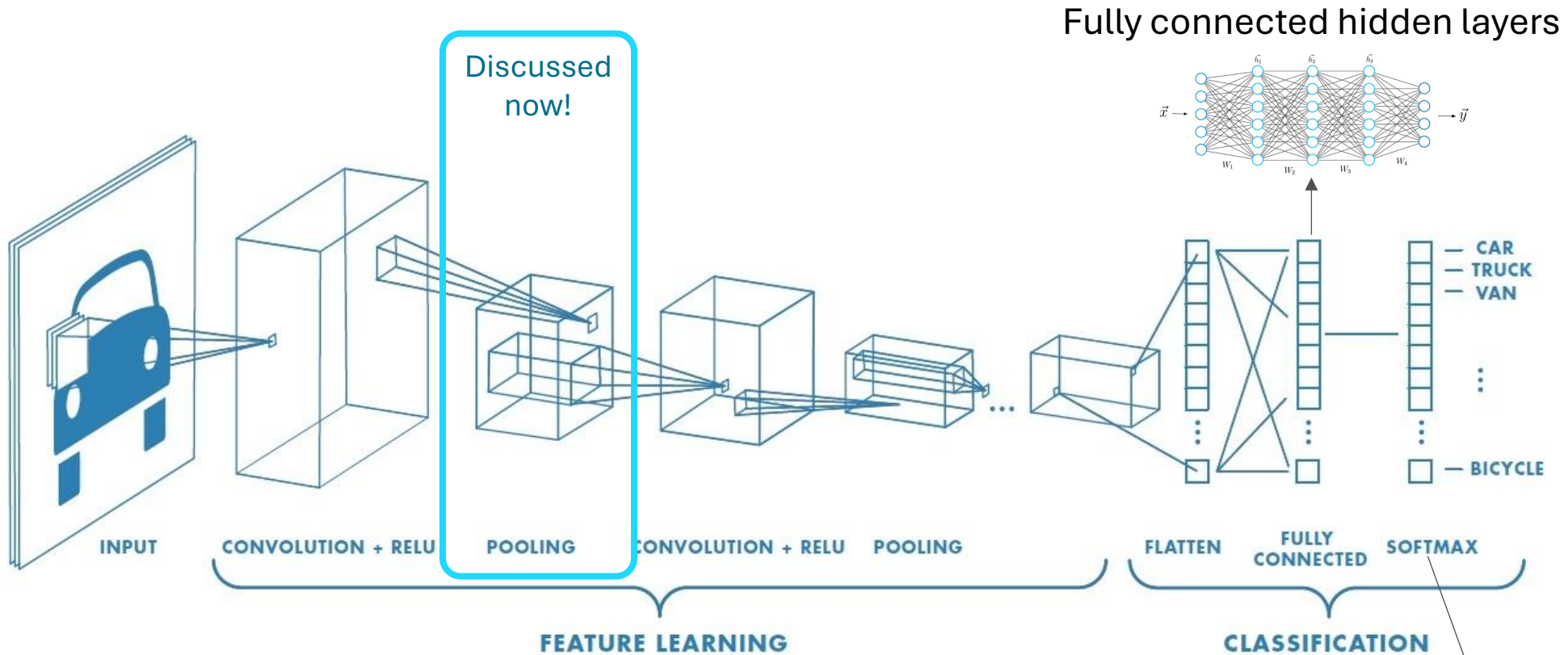


ACTIVATION

As **activation function** for our neurons, we typically use either the **identity** function or **ReLU**. As always, **output of one convolution can act as input for another set of convolutions**, and so on. By using this technique, we are able to define **more complex / abstract features** in each layer of our NN.



STEP BY STEP: POOLING



When used for classification;
for regression the output layer is
just a fully connected layer

STEP BY STEP: POOLING-LAYER (POOL)

PROBLEM: SIZE OF THE CNN

Since there is **no clear rule** how many CONV layers or which filter sizes / amount of filters are needed, there exist several CNN architectures (some guidelines are e.g. defined in [1]).

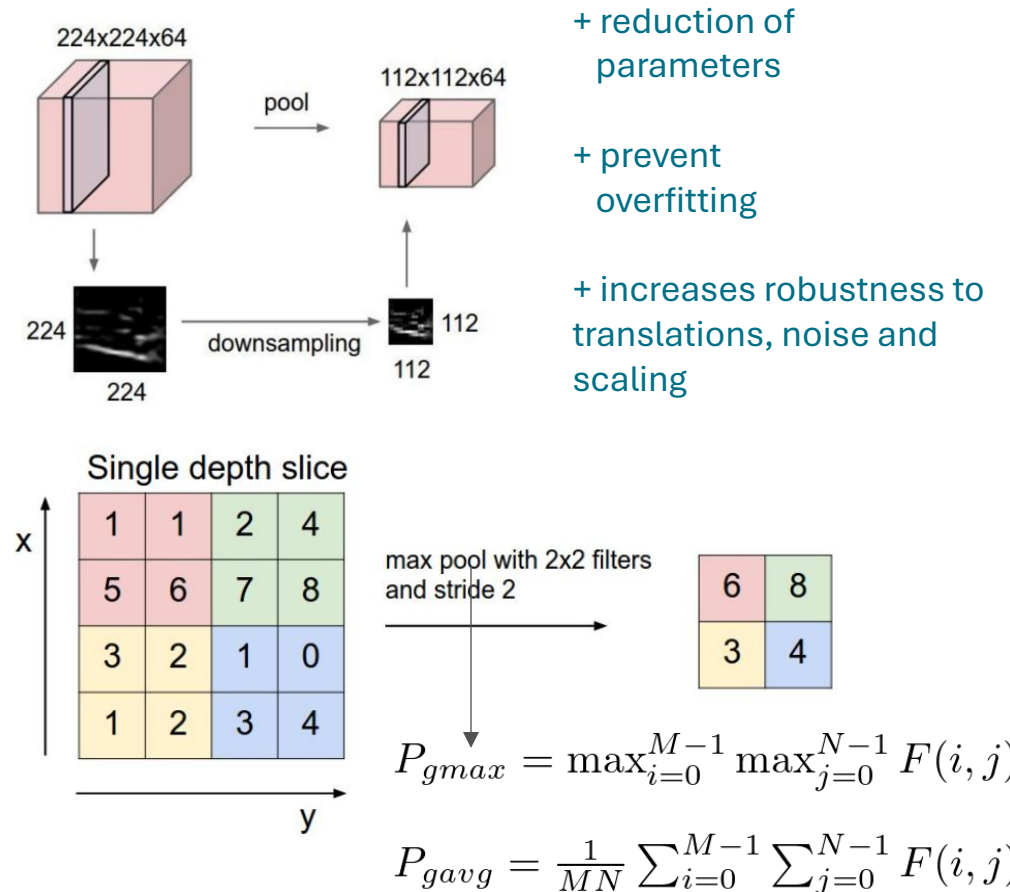
Since our overall problem is **somehow related to regression**, our calculated weights **might suffer from overfitting** with respect to our used training data.

IDEA

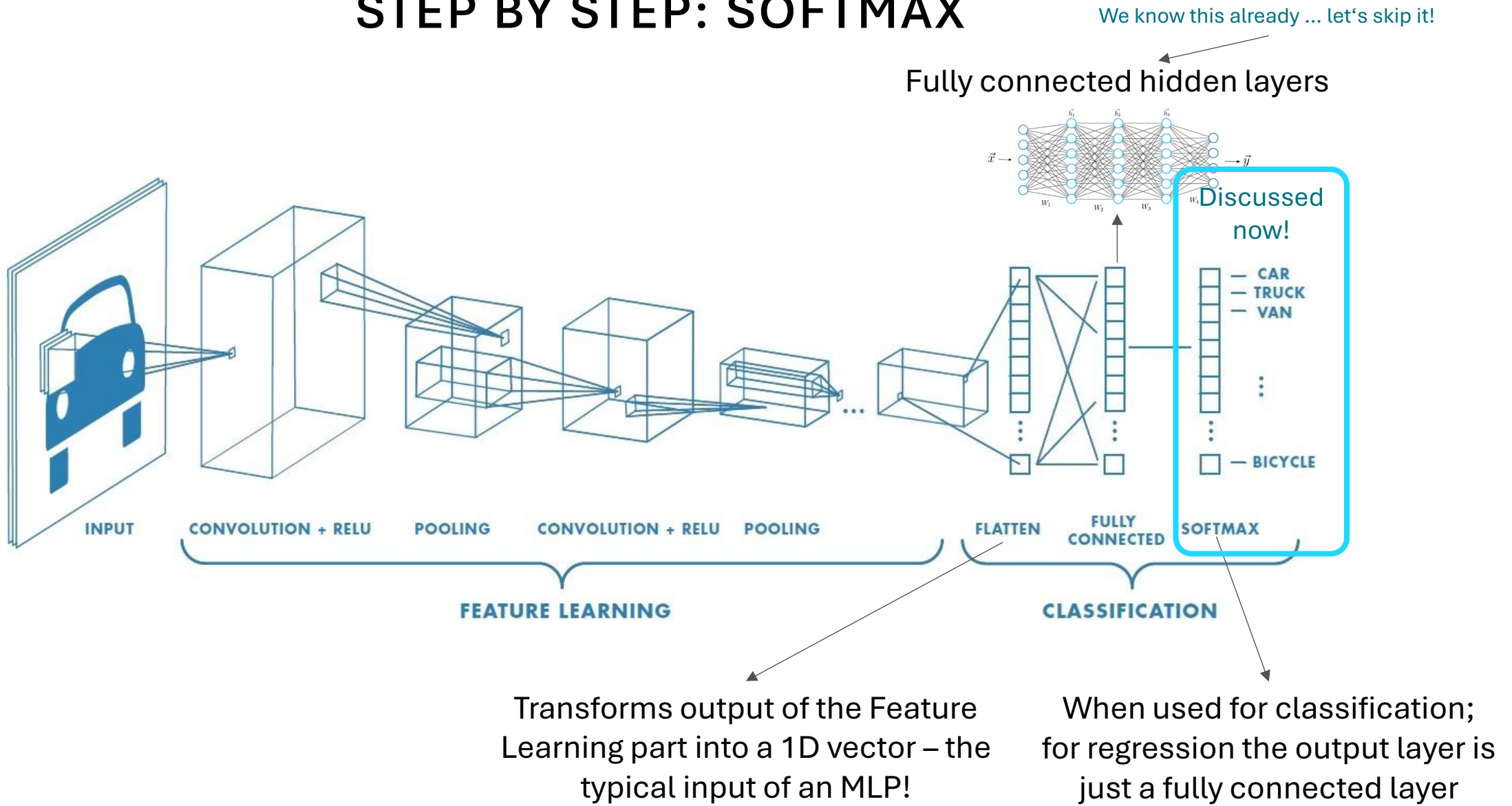
Periodically insert a layer, which **manipulates convolution outputs** in a specific way in order to **prevent overfitting**. In general, this type of operation is called a **pooling**.



In practice, we typically use **max pooling** with a 2x2 filter and stride 2. Other method might be **dropout layers**.



STEP BY STEP: SOFTMAX



STEP BY STEP: SOFTMAX (RECAP)

PROBLEM: A REAL NUMBER IS NOT A CLASS

In classification scenarios, we need to map the output of a neural network (not only of CNNs) to class labels. How can we instead generate the probability of particular classes as output?

IDEA:

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = Softmax-Funktion

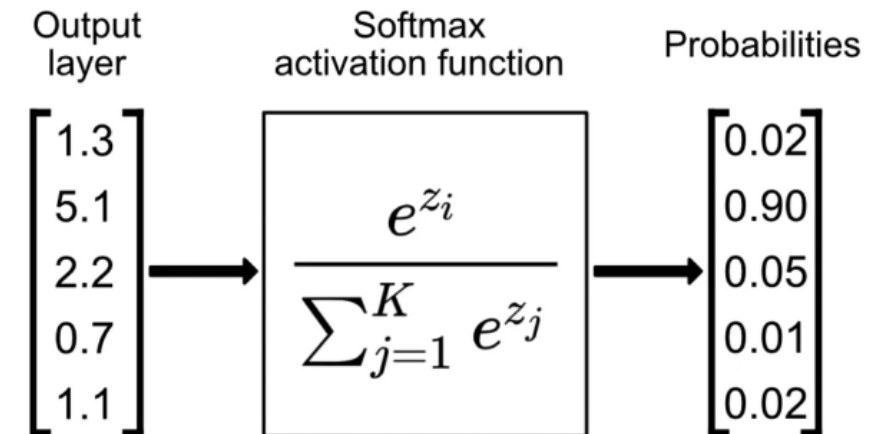
$\vec{z}$ = Eingangsvektor

e^{z_i} = Standard-Exponentialfunktion für Eingangsvektor

K = Anzahl der Klassen in der Multiklassen-Klassifikation

e^{z_j} = Standard-Exponentialfunktion für Ausgangsvektor

e^{z_j} = Standard-Exponentialfunktion für Ausgangsvektor



?

Why not only searching for the highest value to identify the „winning“ class?

!

We use the Softmax layer because it converts the raw, unbounded outputs (logits) into a normalized probability distribution required for training and meaningful probabilistic interpretation.

QUESTIONS?





DONE.

95028 Hof
Alfons-Goppel-Platz 1
Phone +49 9281 409-3000
Fax +49 9281 409-4000
mail@hof-university.de
www.hof-university.de